الاحتراف الكامل

في مكتبات بايثون لعلم البيانات والتعلّم الآلي

الجزء الثالث: مكتبة Pandas

الفصول ١ - ٦: من التعريف بالمكتبة حتى الدوال الأساسية

مرجع احترافي للمبرمجين والمهندسين وعلماء البيانات

جدول المحتويات

جدول المحتويات 2
مقدمة 4
الفصل الأول: التعريف بمكتبة Pandas 5
١.١ لماذا ظهرت Pandas 5
١.٢ المشكلة التي تحلها 5
١.٣ أهم الاستخدامات والشركات 5
١.٤ المميزات والعيوب 5
١.٥ متى تستخدمها ومتى لا 5
١.٦ أشهر البدائل 6
ملخص الفصل 6
الفصل الثاني: التثبيت والإعداد 7
٢.١ أفضل أسلوب استيراد 7
٢.٢ تنظيم المشروع 7
أفضل الممارسات 7
الفصل الثالث: فلسفة التصميم 8
٣.١ DataFrame كجدول مبني فوق مصفوفات NumPy 8
٣.٢ فلسفة "التسمية أولاً" (Label-based) 8
٣.٣ Missing Data كمواطن من الدرجة الأولى 8
٣.٤ البنية الداخلية: BlockManager 8
٣.٥ Backend الحديث: Arrow 9
ملخص الفصل 9
الفصل الرابع: المفاهيم الأساسية 10
٤.١ إنشاء Series وDataFrame 10
٤.٢ Index وMultiIndex 10
٤.٣ الاختيار والتصفية (Selection & Filtering) 10
٤.٤ الترتيب (Sorting) 10
٤.٥ التجميع (GroupBy) 11
٤.٦ الدمج (Merge, Join, Concat) 11
٤.٧ Pivot وMelt 11
٤.٨ القيم المفقودة والمكررة 11
الأخطاء الشائعة 11
تمارين عملية 12
حلول مقترحة (تلميحات) 12

الفصل الخامس: الكائنات وواجهات البرمجة 13
٥.١ خصائص DataFrame الأساسية 13
٥.٢ Method Chaining 13
٥.٣ apply وmap وapplymap 13
٥.٤ nullable dtypes وpd.NA 13
٥.٥ نموذج التنفيذ: Eager مع بعض Lazy الاختياري 14
أفضل الممارسات 14
الفصل السادس: أهم الدوال 15
٦.١ القراءة والكتابة (I/O) 15
٦.٢ التعامل مع التواريخ (DateTime) 15
٦.٣ البيانات الفئوية (Categorical) 15
٦.٤ دوال Series النصية (str accessor) 15
٦.٥ replace وwhere وclip 16
٦.٦ Edge Cases شائعة 16
الأخطاء الشائعة 16
الأسئلة الشائعة 16
تمارين عملية 16
حلول مقترحة (تلميحات) 17
مراجع وروابط رسمية 17
خاتمة الجزء الأول من Pandas 17

مقدمة

تُبنى Pandas مباشرة فوق NumPy، وتضيف إليها ما كانت تفتقده: بنية بيانات جدولية (Tabular) بأعمدة مسمّاة وأنواع بيانات مختلطة، وفهارس (Index) ذكية، وأدوات متخصصة للتعامل مع البيانات الحقيقية الفوضوية: قيم مفقودة، تواريخ، بيانات فئوية، ودمج جداول متعددة. إذا كانت NumPy هي لغة الحساب العددي البحت، فإن Pandas هي لغة 'الجدول' — أقرب ما تكون لجدول بيانات Excel لكن ببرمجية كاملة القوة.

سنعتمد بكثافة على المفاهيم التي أُرسيت في الجزأين السابقين (Views/Copies، Broadcasting، Vectorization)، لأن Pandas تستخدمها داخليًا باستمرار.

الفصل الأول: التعريف بمكتبة Pandas

١.١ لماذا ظهرت Pandas

طوّر Wes McKinney مكتبة Pandas عام ٢٠٠٨ أثناء عمله في شركة AQR Capital Management، وهي شركة استثمار كمّي (Quantitative Finance). كانت المشكلة أن بايثون تفتقر لأداة قوية لتحليل البيانات المالية الجدولية (سلاسل زمنية، جداول بأعمدة مختلفة الأنواع) بالمستوى الذي توفره أدوات مثل R وجداول Excel، رغم قوة NumPy في الحساب العددي البحت.

الاسم Pandas مشتق من مصطلح Panel Data، وهو مصطلح اقتصادي قياسي يشير إلى بيانات متعددة الأبعاد تجمع بين المقطع العرضي (Cross-section) والسلسلة الزمنية. منذ إصدارها كمشروع مفتوح المصدر عام ٢٠٠٩، أصبحت المعيار الفعلي (De facto standard) لتحليل البيانات الجدولية في بايثون.

١.٢ المشكلة التي تحلها

- التعامل مع بيانات جدولية بأعمدة مسمّاة وأنواع مختلطة (نصوص، أرقام، تواريخ) في بنية واحدة.
- التعامل الذكي مع القيم المفقودة (NaN) دون أن يتوقف الكود.
- قراءة وكتابة صيغ ملفات متعددة (CSV, Excel, JSON, Parquet, SQL) بواجهة موحدة.
- عمليات تجميع (GroupBy) ودمج (Merge/Join) شبيهة بلغة SQL لكن داخل بايثون مباشرة.
- التعامل المتخصص مع السلاسل الزمنية (DateTime) الشائعة في التمويل والاقتصاد.

١.٣ أهم الاستخدامات والشركات

تُستخدم Pandas في كل مرحلة تقريبًا من مراحل تحليل البيانات: التنظيف، الاستكشاف، التحويل، وتحضير البيانات للنمذجة. تعتمد عليها شركات التمويل (حيث نشأت أصلاً)، وشركات التقنية الكبرى، والمؤسسات البحثية والحكومية. من أشهر المشاريع التي تعتمد عليها: Scikit-learn (لتحضير البيانات قبل التدريب)، وStatsmodels، وأدوات لوحات المعلومات مثل Streamlit وDash.

١.٤ المميزات والعيوب

الجانب	التفاصيل
مميزات	واجهة تعبيرية غنية، تكامل ممتاز مع NumPy وMatplotlib، دعم قوي للسلاسل الزمنية، قراءة/كتابة صيغ ملفات متعددة
عيوب	استهلاك ذاكرة أعلى من NumPy الخام، أداء أبطأ على البيانات الضخمة جدًا مقارنة بأدوات مثل Polars، بعض العمليات (apply) مع دالة بايثون) بطيئة لأنها غير متجهة بالكامل
حدود	غير مصممة أصلاً للتوازي التلقائي أو البيانات الأكبر من الذاكرة (تحتاج Dask أو Polars لتلك الحالات)

١.٥ متى تستخدمها ومتى لا

القرار	الحالة
استخدم Pandas	بيانات جدولية متوسطة الحجم تتسع في الذاكرة

القرار	الحالة
استخدم Dask أو Polars أو Spark	بيانات ضخمة جدًا لا تتسع بالذاكرة
فكّر في Polars (أسرع وأكفأ ذاكرة)	أداء قصوى الأهمية على بيانات كبيرة نسبيًا
استخدم NumPy مباشرة بدل تحويلها لـ DataFrame	حسابات عددية بحتة على مصفوفات متجانسة

١.٦ أشهر البدائل

- Polars: مكتبة حديثة مكتوبة بلغة Rust، أسرع بشكل ملحوظ وأكفأ في استهلاك الذاكرة، وتدعم Lazy Evaluation.
- Dask DataFrame: يوزّع بيانات Pandas الضخمة على أجزاء (Partitions) ويعالجها بالتوازي.
- PySpark: لمعالجة بيانات ضخمة موزّعة على عناقيد (Clusters) كاملة.
- Modin: يحاول تسريع Pandas بواجهة برمجية متطابقة تمامًا مع تنفيذ موازٍ خلف الكواليس.

ملخص الفصل

- Pandas وُلدت في بيئة تمويل كمّي لحل مشكلة التعامل مع البيانات الجدولية المختلطة الأنواع.
- تُبنى فوق NumPy وتضيف بنى بيانات جدولية (DataFrame) وفهارس ذكية وأدوات تنظيف بيانات.
- الأنسب للبيانات المتوسطة الحجم؛ للبيانات الضخمة جدًا انظر Polars أو Dask.

الفصل الثاني: التثبيت والإعداد

```
pip install pandas
conda install pandas

# مع دعم صيغ إضافية (Excel, Parquet)
pip install "pandas[excel,parquet]"
```

مثل NumPy، يُفضَّل تثبيت Pandas داخل بيئة افتراضية معزولة لكل مشروع، وتحديد الإصدار في requirements.txt، خصوصًا أن الانتقال من Pandas 1.x إلى ٢.x غيّر السلوك الافتراضي لبعض العمليات (مثل معالجة القيم المفقودة في الأعمدة الفئوية) ودعم Backend بديل لـ Arrow.

٢.١ أفضل أسلوب استيراد

```
import pandas as pd
import numpy as np   # غالبًا ما تُستخدمان معًا
```

٢.٢ تنظيم المشروع

```
project/
├── data/raw/            # ملفات CSV/Excel الأصلية (لا تُعدَّل أبدًا)
├── data/processed/      # نتائج التنظيف كملفات Parquet
├── src/
│   ├── load.py          # pd.read_csv / read_excel...
│   ├── clean.py         # معالجة القيم المفقودة والتكرار
│   └── transform.py     # groupby / merge / pivot
└── notebooks/
```

ملاحظة احترافية: يُفضَّل حفظ البيانات المُعالَجة بصيغة Parquet بدل CSV، لأنها تحافظ على أنواع البيانات (dtype) بدقة وأسرع في القراءة والكتابة وأصغر حجمًا.

أفضل الممارسات

- استخدم import pandas as pd دائمًا، وimport numpy as np معها.
- لا تُعدّل بيانات data/raw أبدًا؛ اكتب النتائج المُعالَجة في مسار منفصل.
- فضّل Parquet على CSV للتخزين الوسيط بين مراحل خط الأنابيب.

الفصل الثالث: فلسفة التصميم

٣.١ DataFrame كجدول مبني فوق مصفوفات NumPy

الفكرة الجوهرية: DataFrame هو في الأساس مجموعة من أعمدة Series، وكل Series هو -من الداخل- مصفوفة NumPy واحدة البُعد (أو مصفوفة Arrow في الإصدارات الحديثة) مرفقة بفهرس (Index). هذا يعني أن كل عمود يمكن أن يكون بنوع بيانات مختلف تمامًا عن العمود المجاور، بينما يبقى كل عمود بمفرده متجانسًا (Homogeneous) من الداخل — وهنا يظهر التكامل العميق مع فلسفة NumPy القائمة على التجانس.

البنية	الوصف
Series	عمود واحد أحادي البُعد + فهرس، شبيه بمصفوفة NumPy مع تسميات
DataFrame	جدول ثنائي الأبعاد = قاموس من أعمدة Series تتشارك نفس الفهرس (Index)
Index	بنية تسمية الصفوف (أو الأعمدة)، يمكن أن تكون أرقامًا أو تواريخ أو نصوصًا أو متعددة المستويات

٣.٢ فلسفة "التسمية أولاً" (Label-based)

بينما NumPy تتعامل بالفهرسة الموضعية البحتة (Position-based)، تتبنى Pandas فلسفة "التسمية أولاً": كل عملية بحث أو محاذاة (Alignment) تتم افتراضيًا حسب تسمية الفهرس وليس موضعه. هذا يفسّر لماذا عند جمع Series باستخدام +، تُحاذي Pandas العناصر حسب تسمية الفهرس تلقائيًا، حتى لو كانت الفهارس بترتيب مختلف.

```
a = pd.Series([1, 2, 3], index=['x', 'y', 'z'])
b = pd.Series([10, 20, 30], index=['z', 'y', 'x'])
a + b
# x    31   <- تمت المحاذاة حسب التسمية وليس الموضع
# y    22
# z    13
```

٣.٣ Missing Data كمواطن من الدرجة الأولى

على عكس NumPy التي لا تملك تمثيلاً موحدًا وطبيعيًا للقيم المفقودة (تضطر لاستخدام NaN من float فقط)، صُمّمت Pandas منذ البداية لتتعامل مع القيم المفقودة كحالة طبيعية متوقعة في أي عمود، عبر NaN لـ float وNaT للتواريخ ومؤخرًا pd.NA الموحّد عبر كل الأنواع بما فيها الأعداد الصحيحة القابلة للـ Nullable.

٣.٤ البنية الداخلية: BlockManager

داخليًا، لا تخزّن Pandas كل عمود كمصفوفة منفصلة تمامًا؛ بل تجمع الأعمدة المتشابهة في dtype داخل كتل (Blocks) متجاورة في الذاكرة لتحسين الأداء، عبر مكوّن داخلي يُسمى BlockManager. هذا التفصيل غير مرئي للمستخدم العادي، لكنه يفسّر لماذا العمليات على عمود واحد أحيانًا أسرع من المتوقع، ولماذا تغيير dtype عمود قد يُعيد تنظيم الذاكرة بالكامل داخليًا.

٣.٥ Backend الحديث: Arrow

بدءًا من Pandas 2.0، أصبح بالإمكان استخدام Apache Arrow كـ backend بديل لتخزين البيانات بدل NumPy التقليدي، خاصة للنصوص، مما يحسّن الأداء واستهلاك الذاكرة وينسجم بشكل أفضل مع أنظمة بيانات حديثة أخرى (مثل Polars وSpark).

```
df = pd.read_csv('data.csv', dtype_backend='pyarrow')
```

ملخص الفصل

- DataFrame هو مجموعة أعمدة Series تتشارك فهرسًا واحدًا؛ كل عمود متجانس داخليًا لكن الأعمدة قد تختلف في النوع.
- الفلسفة المحورية هي "التسمية أولاً": المحاذاة والعمليات تتم حسب تسمية الفهرس لا موضعه.
- القيم المفقودة مواطن من الدرجة الأولى في التصميم، عبر NaN وNaT وpd.NA.
- Backend Arrow الحديث بديل أكفأ لتخزين NumPy التقليدي في بعض الحالات.

الفصل الرابع: المفاهيم الأساسية

٤.١ إنشاء Series وDataFrame

```
import pandas as pd

s = pd.Series([10, 20, 30], index=['a', 'b', 'c'], name='sales')

df = pd.DataFrame({
    'name': ['Ali', 'Sara', 'Omar'],
    'age': [25, 30, 22],
    'city': ['Cairo', 'Giza', 'Cairo'],
})
```

٤.٢ Index وMultiIndex

Index هو ما يمنح Pandas قوتها التعبيرية؛ يمكن أن يكون بسيطًا (أرقام متسلسلة افتراضية) أو مخصصًا (نصوص، تواريخ) أو متعدد المستويات (MultiIndex) لتمثيل بيانات هرمية مثل (سنة، شهر) أو (دولة، مدينة).

```
df.set_index('name', inplace=True)      # جعل عمود name هو الفهرس

arrays = [['2023', '2023', '2024'], ['Q1', 'Q2', 'Q1']]
multi_idx = pd.MultiIndex.from_arrays(arrays, names=['year', 'quarter'])
s2 = pd.Series([100, 150, 200], index=multi_idx)
s2['2023']       # كل قيم 2023 عبر الفهرسة الجزئية
```

٤.٣ الاختيار والتصفية (Selection & Filtering)

الأداة	النوع	مثال
df['col']	اختيار عمود	يُرجع Series
df[['a','b']]	اختيار عدة أعمدة	يُرجع DataFrame
df.loc[label]	فهرسة بالتسمية	df.loc['Ali']
df.iloc[pos]	فهرسة بالموضع	df.iloc[0]
df[df['age'] > 25]	فهرسة منطقية (Boolean)	تصفية صفوف بشرط

```
df.loc[df['city'] == 'Cairo', 'age']    # فقط Cairo لكل صفوف age عمود
df.iloc[0:2, 0:1]                       # أول صفين، أول عمود، بالموضع
```

الفرق الجوهري: loc تعتمد على التسمية (Label) وتشمل الحد الأخير في التقطيع (Inclusive)، بينما iloc تعتمد على الموضع الرقمي (Position) ولا تشمل الحد الأخير (Exclusive)، تمامًا كسلوك NumPy وقوائم بايثون.

٤.٤ الترتيب (Sorting)

```
df.sort_values('age', ascending=False)
df.sort_index()
df.sort_values(['city', 'age'], ascending=[True, False])   # ترتيب متعدد المستويات
```

٤.٥ التجميع (GroupBy)

GroupBy يتبع نمط Split-Apply-Combine: تقسيم البيانات حسب مفتاح، تطبيق دالة تجميعية على كل مجموعة، ثم دمج النتائج في بنية واحدة. هذا النمط مستوحى مباشرة من SQL's GROUP BY لكنه أكثر مرونة وتعبيرًا.

```
df.groupby('city')['age'].mean()
df.groupby('city').agg(
    avg_age=('age', 'mean'),
    total=('age', 'count'),
)
```

٤.٦ الدمج (Merge, Join, Concat)

الاستخدام	الدالة
تكديس جداول عموديًا أو أفقيًا	pd.concat([df1, df2])
دمج شبيه بـ SQL JOIN حسب عمود مشترك	pd.merge(df1, df2, on='key')
دمج حسب الفهرس (Index) بدل عمود	df1.join(df2)

```
orders = pd.DataFrame({'order_id':[1,2], 'customer_id':[10,20]})
customers = pd.DataFrame({'customer_id':[10,20], 'name':['Ali','Sara']})
pd.merge(orders, customers, on='customer_id', how='left')
```

٤.٧ Pivot وMelt

```
df.pivot_table(values='sales', index='city', columns='quarter', aggfunc='sum')

wide = pd.DataFrame({'id':[1,2], 'q1':[10,20], 'q2':[15,25]})
wide.melt(id_vars='id', var_name='quarter', value_name='sales')  # من عريض إلى طويل
```

pivot_table تحوّل بيانات طويلة (Long) إلى عريضة (Wide)، بينما melt تعمل العكس. هذا التحويل بين الشكلين من أكثر العمليات تكرارًا في تحضير البيانات للتحليل أو الرسم البياني.

٤.٨ القيم المفقودة والمكررة

```
df.isna().sum()                  # عدد القيم المفقودة لكل عمود
df.dropna(subset=['age'])        # حذف الصفوف التي تحتوي NaN في age
df.fillna({'age': df['age'].mean()})  # تعبئة القيم المفقودة بالمتوسط
df.duplicated().sum()            # عدد الصفوف المكررة
df.drop_duplicates()
```

الأخطاء الشائعة

- الخلط بين loc وiloc، خصوصًا في التقطيع حيث loc تشمل الحد الأخير وiloc لا تشمله.
- استخدام Chained Indexing مثل df[df.a>0]['b'] = 1 الذي قد لا يعدّل DataFrame الأصلي ويصدر SettingWithCopyWarning.
- نسيان معامل how في pd.merge والاعتماد على القيمة الافتراضية inner دون قصد، مما قد يفقد صفوفًا بصمت.

تمارين عملية

١. أنشئ DataFrame لعشرة موظفين بأعمدة (الاسم، القسم، الراتب)، ثم احسب متوسط الراتب لكل قسم عبر .groupby

٢. أنشئ جدولين (موظفين وأقسام) وادمجهما عبر merge بمفتاح مشترك.

٣. حوّل جدول مبيعات بصيغة عريضة (عمود لكل شهر) إلى صيغة طويلة باستخدام .melt

حلول مقترحة (تلميحات)

١. df.groupby('القسم')['الراتب'].mean()

٢. pd.merge(employees, departments, on='dept_id', how='left')

٣. df.melt(id_vars='product', var_name='month', value_name='sales')

الفصل الخامس: الكائنات وواجهات البرمجة

٥.١ خصائص DataFrame الأساسية

الخاصية	الوصف
df.shape	عدد الصفوف والأعمدة
df.columns	أسماء الأعمدة
df.index	الفهرس
df.dtypes	نوع بيانات كل عمود
df.values	تحويل إلى مصفوفة NumPy (يفقد أسماء الأعمدة وأنواعها المختلطة)
df.info()	ملخص شامل: الأعمدة، الأنواع، القيم غير المفقودة، الذاكرة

٥.٢ Method Chaining

أسلوب Method Chaining هو الأسلوب الاحترافي الموصى به في Pandas: تسلسل عمليات متعددة في تعبير واحد مقروء بدل متغيرات وسيطة كثيرة، خصوصًا عند استخدام الأقواس لتقسيم السلسلة على عدة أسطر.

```
result = (
    df
    .query('age > 20')
    .groupby('city')
    .agg(avg_age=('age', 'mean'))
    .sort_values('avg_age', ascending=False)
)
```

٥.٣ apply وmap وapplymap

الدالة	تعمل على	الاستخدام
Series.map(func)	عنصر بعنصر في Series	تحويل قيم عمود واحد
DataFrame.apply(func, axis)	صف أو عمود كامل	دالة تُطبَّق على كل صف/عمود
DataFrame.map(func)	كل عنصر في DataFrame	(الاسم الحديث بدل applymap المهجورة)

```
df['age_group'] = df['age'].map(lambda x: 'شاب' if x < 30 else 'كبير')
df['full_info'] = df.apply(lambda row: f"{row['name']} - {row['city']}", axis=1)
```

تنبيه أداء: apply مع دالة بايثون عادية ليست متجهة (Vectorized) — فهي تستدعي حلقة بايثون داخليًا، وبالتالي أبطأ بكثير من عملية Pandas مدمجة معادلة. استخدمها فقط عندما لا يوجد بديل متجهة.

٥.٤ nullable dtypes وpd.NA

```
s = pd.Series([1, 2, None], dtype='Int64')   # 'I' كبيرة = Nullable Integer
s   # 0    1
    # 1    2
    # 2   <NA>
```

dtype القياسي int64 (بحرف صغير) في NumPy لا يدعم NaN إطلاقًا، لذلك كان أي عمود صحيح فيه قيمة مفقودة يُحوَّل قسرًا إلى float64. الأنواع Nullable الحديثة مثل Int64 تحل هذه المشكلة بدعم pd.NA مباشرة دون فقدان دقة الأعداد الصحيحة.

٥.٥ نموذج التنفيذ: Eager مع بعض Lazy الاختياري

مثل NumPy، تتبع Pandas التقليدية نموذج تنفيذ فوري (Eager). لكن مع ظهور Backend الخاص بـ Arrow ومحركات مثل Polars، بدأ الاتجاه نحو Lazy Evaluation (بناء خطة تنفيذ أولاً ثم تحسينها وتنفيذها لاحقًا) كخيار أداء في السياقات الحديثة، رغم أن Pandas الأساسية نفسها ما زالت Eager بالكامل.

أفضل الممارسات

- فضّل العمليات المدمجة في Pandas (groupby, merge, vectorized string methods) على apply مع دالة بايثون يدوية.
- استخدم Method Chaining للتعبيرات متعددة الخطوات لتحسين القراءة.
- استخدم أنواع Nullable (Int64, boolean) عند وجود قيم مفقودة في أعمدة أعداد صحيحة أو منطقية.

الفصل السادس: أهم الدوال

٦.١ القراءة والكتابة (I/O)

الصيغة	الدالة
CSV	pd.read_csv / df.to_csv
Excel (يتطلب openpyxl)	pd.read_excel / df.to_excel
JSON	pd.read_json / df.to_json
Parquet (يتطلب pyarrow)	pd.read_parquet / df.to_parquet
قواعد بيانات SQL (يتطلب SQLAlchemy)	pd.read_sql / df.to_sql

```
df = pd.read_csv('sales.csv', parse_dates=['order_date'])
df.to_parquet('sales_clean.parquet', index=False)

import sqlalchemy
engine = sqlalchemy.create_engine('sqlite:///db.sqlite')
df.to_sql('sales', engine, if_exists='replace', index=False)
```

٦.٢ التعامل مع التواريخ (DateTime)

```
df['order_date'] = pd.to_datetime(df['order_date'])
df['year'] = df['order_date'].dt.year
df['weekday'] = df['order_date'].dt.day_name()

monthly = df.set_index('order_date').resample('ME')['sales'].sum()  # تجميع شهري
```

محور dt هو المدخل الموحّد لكل خصائص ووظائف التواريخ في Pandas (السنة، الشهر، اسم اليوم، الفرق بين تاريخين، إلخ)، وresample هو أداة GroupBy المتخصصة للسلاسل الزمنية.

٦.٣ البيانات الفئوية (Categorical)

```
df['city'] = df['city'].astype('category')
df['city'].cat.categories
```

تحويل عمود نصي متكرر القيم (مثل أسماء المدن أو الفئات) إلى dtype='category' يقلل استهلاك الذاكرة بشكل كبير جدًا (لأن Pandas تخزّن أرقامًا صحيحة داخليًا بدل تكرار النص الكامل)، ويسرّع عمليات groupby وsort_values على ذلك العمود.

٦.٤ دوال Series النصية (str accessor)

```
df['name'].str.upper()
df['name'].str.contains('Ali')
df['email'].str.split('@').str[0]
```

محور str يوفّر عمليات نصية متجهة (Vectorized) تعمل على العمود بأكمله دفعة واحدة، بدل الحاجة لحلقة for أو apply يدوي، وهي أسرع بكثير وأكثر توافقًا مع أسلوب Pandas الاصطلاحي.

٦.٥ replace وwhere وclip

```
df['status'].replace({'y': 'yes', 'n': 'no'})
df['age'].where(df['age'] > 0, other=0)     # استبدال القيم السالبة بصفر
df['age'].clip(lower=0, upper=100)           # تقييد القيم ضمن نطاق
```

٦.٦ Edge Cases شائعة

السلوك	الحالة
دائمًا False، حتى NaN == NaN تُرجع False	مقارنة NaN بأي قيمة
ينتج ضربًا ديكارتيًا جزئيًا (Cartesian product) قد يُضخّم عدد الصفوف بشكل غير متوقع	merge بمفاتيح مكررة في الجانبين
تحذير يظهر عند التعديل على نسخة قد تكون View أو Copy غير مؤكدة	SettingWithCopyWarning
قد تُستنتج أنواع خاطئة (مثل أرقام كنصوص بسبب فاصلة عشرية محلية)	قراءة CSV بدون تحديد dtype

```
# الحل الصحيح لتجنب SettingWithCopyWarning
subset = df[df['age'] > 25].copy()   # نسخة صريحة وواضحة النية
subset['age_group'] = 'adult'        # آمن تمامًا الآن
```

الأخطاء الشائعة

- استخدام apply لعمليات نصية بدل محور str المتجهة الأسرع بكثير.
- تجاهل SettingWithCopyWarning بدل فهم سببه وإضافة .copy() صراحة عند الحاجة.
- عدم تحديد 'dtype='category للأعمدة الفئوية عالية التكرار في بيانات ضخمة، مما يهدر ذاكرة كبيرة بلا داعٍ.

الأسئلة الشائعة

س: متى أستخدم category بدل object للنصوص؟

ج: عندما يكون عدد القيم الفريدة صغيرًا نسبيًا مقارنة بعدد الصفوف (مثل المدن أو الفئات)، فإن category توفّر ذاكرة كبيرة وتسرّع العمليات.

س: هل يجب استبدال Pandas بـ Polars دائمًا؟

ج: لا؛ Pandas أنضج توثيقًا وأوسع تكاملاً مع مكتبات أخرى مثل Scikit-learn وMatplotlib. Polars يستحق التفكير عند وجود بيانات ضخمة جدًا أو حاجة لأداء أعلى.

تمارين عملية

١. اقرأ ملف CSV يحتوي عمود تاريخ، حوّله إلى datetime، واستخرج المبيعات الشهرية المجمّعة عبر resample.

٢. حوّل عمود مدينة متكرر القيم إلى category وقارن حجم الذاكرة قبل وبعد عبر df.memory_usage(deep=True).

٣. استخدم محور str لاستخراج اسم النطاق (Domain) من عمود بريد إلكتروني.

حلول مقترحة (تلميحات)

١. pd.to_datetime ثم ()set_index().resample('ME').sum

٢. df['city'].memory_usage(deep=True) قبل وبعد astype('category')

٣. df['email'].str.split('@').str[1]

مراجع وروابط رسمية

- التوثيق الرسمي: https://pandas.pydata.org/docs/
- دليل المستخدم: https://pandas.pydata.org/docs/user_guide/index.html
- الكود المصدري: https://github.com/pandas-dev/pandas

خاتمة الجزء الأول من Pandas

غطّينا حتى الآن تاريخ Pandas وفلسفتها ومفاهيمها الجوهرية (Series، DataFrame، Index، GroupBy، Merge)، وأهم دوالها اليومية. الجزء الثاني سيتناول أمثلة عملية متقدمة، الأداء، التصحيح، أفضل الممارسات، ومشاريع مصغّرة كاملة باستخدام Pandas فقط.